AWS Three-Tier Architecture Project

AWS Three-Tier Student Application

1. Project Introduction

A three-tier architecture splits an application into three layers: a **presentation** tier (what the user sees), an **application** tier (business logic) and a **data** tier (database). This project builds a **Student Registration** web application on AWS in the Asia Pacific (Mumbai) region (ap-south-1). Students can be registered through a web form, and the saved records are shown in a Students List.

AWS services and software used: Amazon VPC (subnets, route tables, Internet Gateway, NAT Gateway), Amazon EC2 with security groups, Amazon RDS (MariaDB), EC2 Instance Connect and Apache Tomcat 9.0.120.

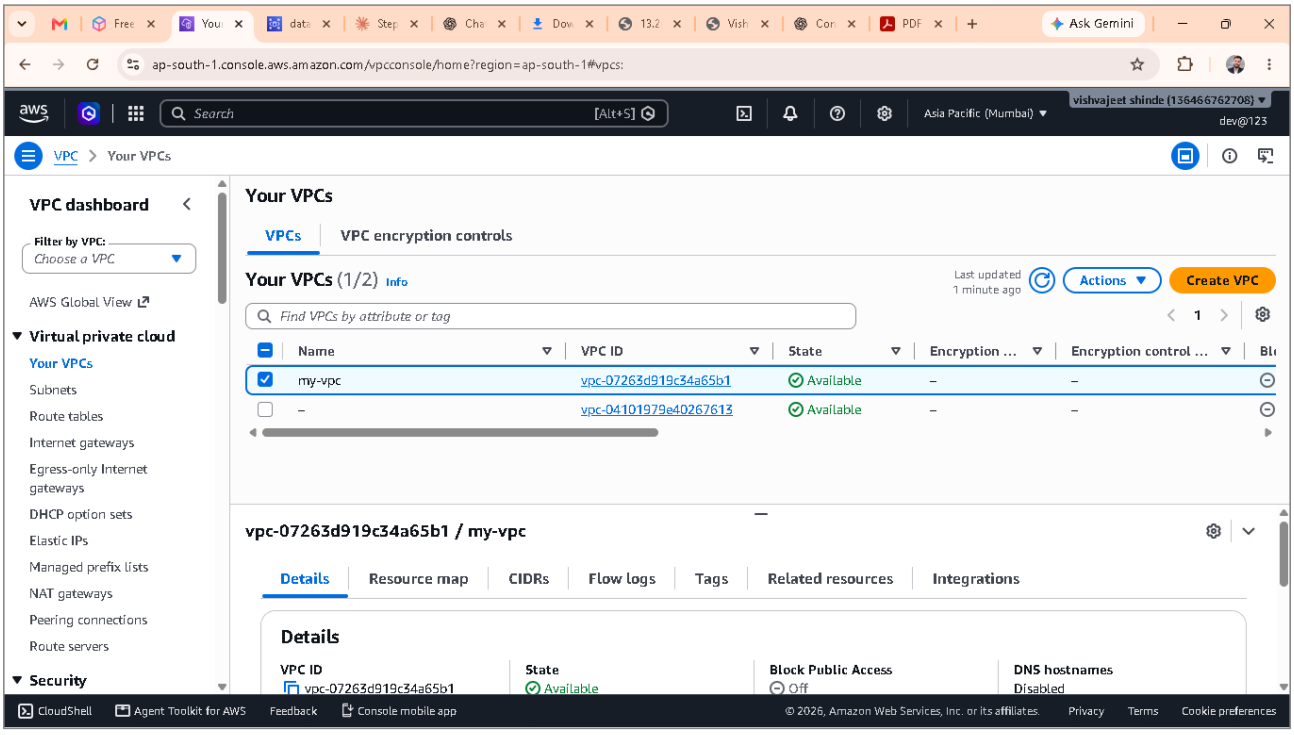
Part A — Network Setup (VPC)

Step 1 — VPC Foundation

What it shows: The VPC dashboard with the project VPC selected.

Details visible in the screenshot:

- Name: **my-vpc**; VPC ID: vpc-07263d919c34a65b1
- State: Available; Block Public Access: Off; DNS hostnames: Disabled
- A second, unnamed VPC (vpc-04101979e40267613) also exists in the account
- IPv4 CIDR: Not visible in the provided screenshot

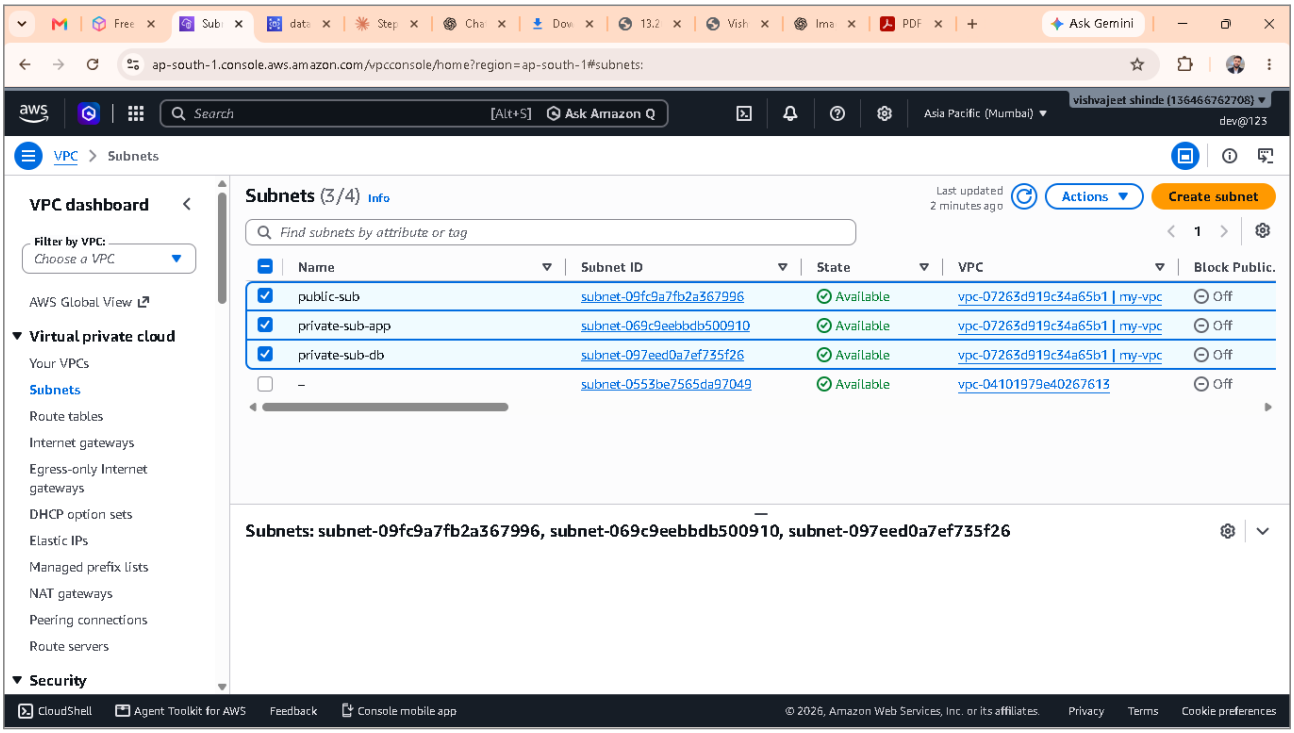


Step 2 — Subnets (Public and Private)

What it shows: The three project subnets that belong to my-vpc.

Details visible in the screenshot:

- public-sub — subnet-09fc9a7fb2a367996
- private-sub-app — subnet-069c9eebbdb500910 (application tier)
- private-sub-db — subnet-097eed0a7ef735f26 (database tier)
- All three: State Available; Block Public Access: Off; VPC: my-vpc
- Subnet CIDRs and Availability Zones: Not visible in the provided screenshot

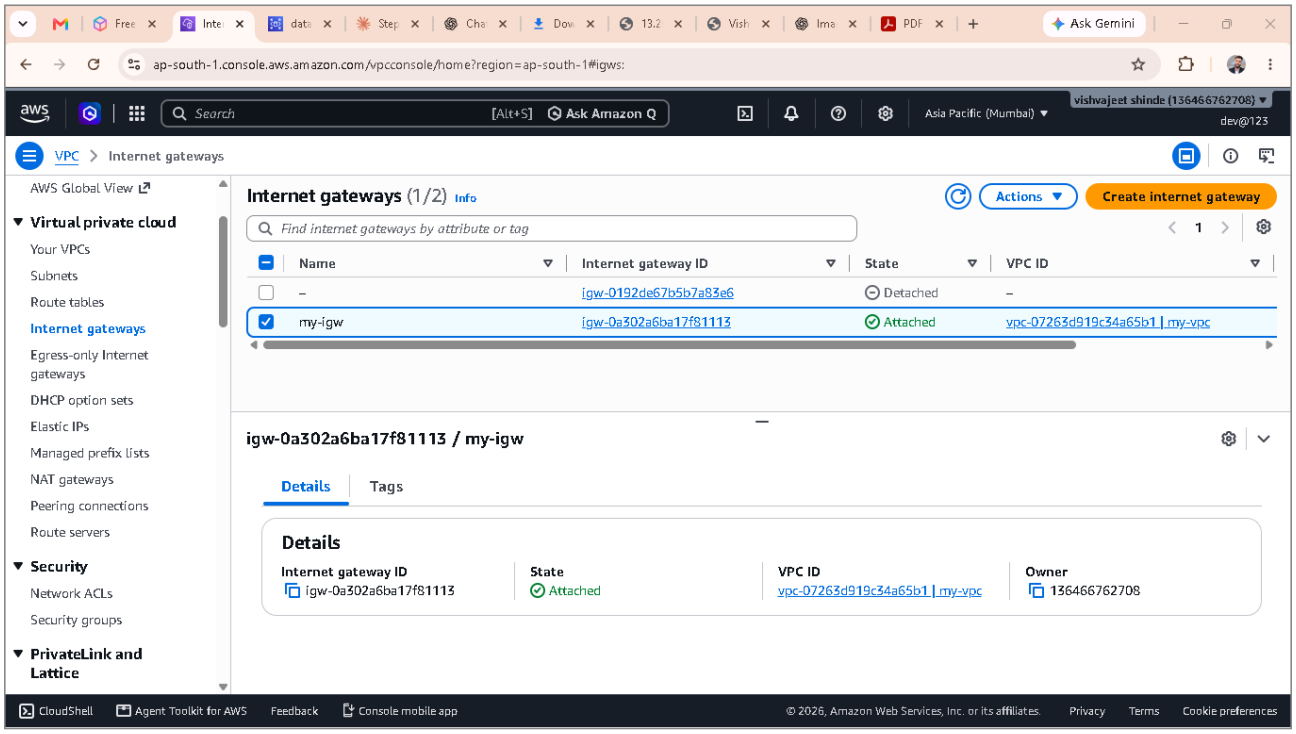


Step 3 — Internet Gateway

What it shows: The Internet Gateway attached to the project VPC. It gives the VPC a path to the public internet when routing and public addressing allow it.

Details visible in the screenshot:

- Name: **my-igw**; ID: igw-0a302a6ba17f81113
- State: Attached
- Attached to: my-vpc / vpc-07263d919c34a65b1
- A second, unnamed gateway (igw-0192de67b5b7a83e6) is shown as Detached

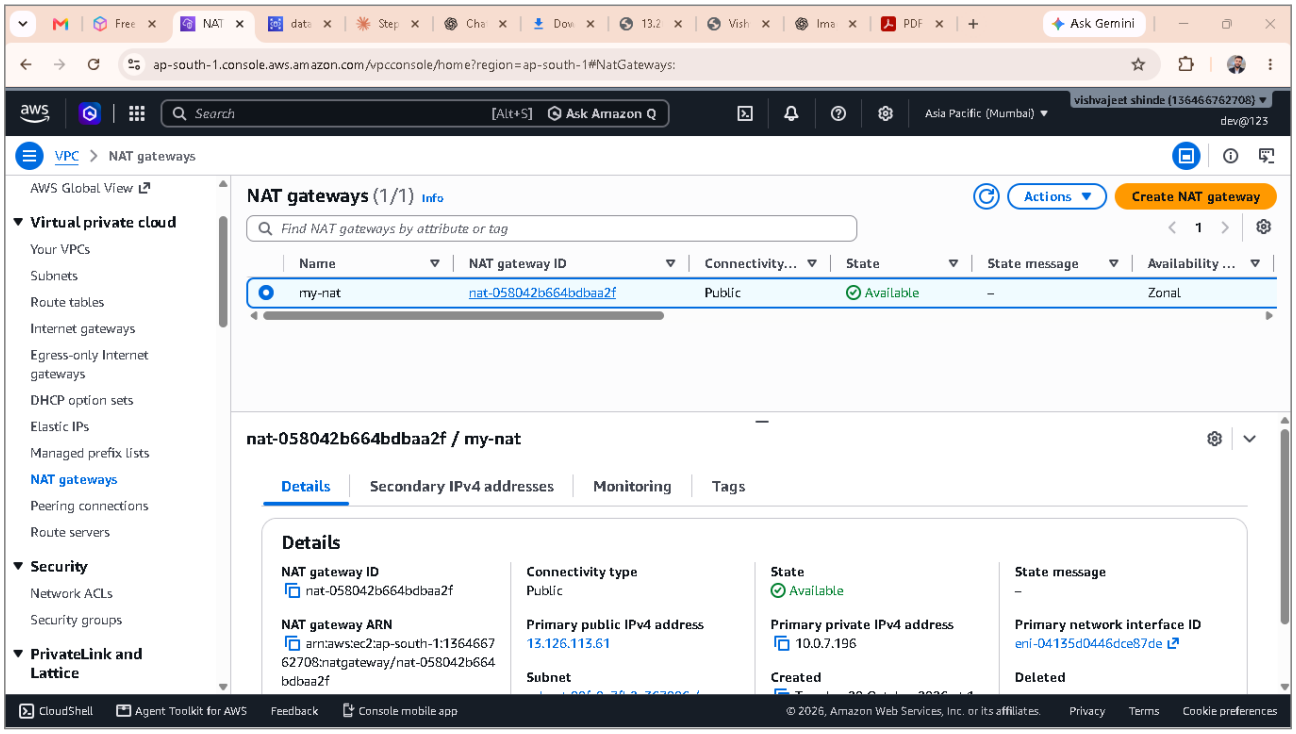


Step 4 — NAT Gateway

What it shows: A public NAT Gateway for the VPC. It lets private resources reach the internet for outbound traffic without becoming public themselves.

Details visible in the screenshot:

- Name: **my-nat**; ID: nat-058042b664bdbaa2f
- Connectivity type: Public; State: Available; Availability: Zonal
- Primary public IPv4: 13.126.113.61; Primary private IPv4: 10.0.7.196
- Primary network interface ID: eni-04135d0446dce87de
- Subnet and creation time: Only partly visible in the provided screenshot

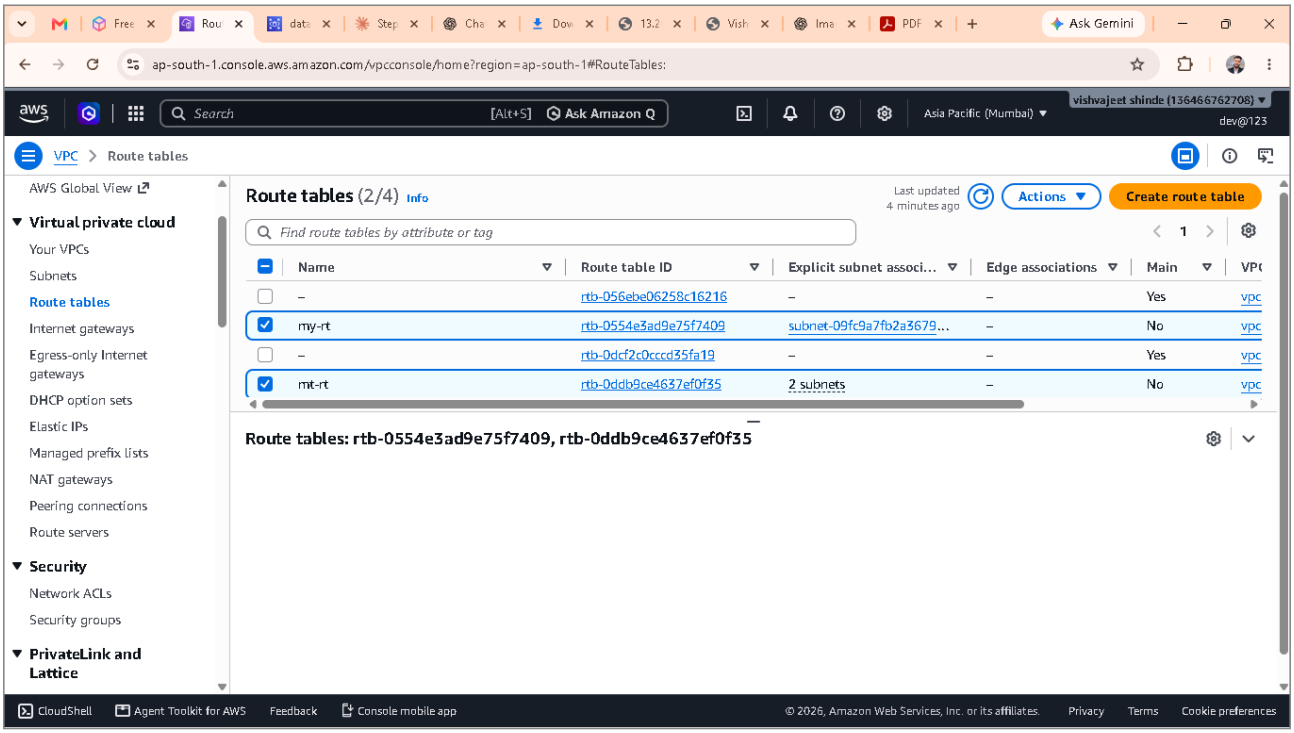


Step 5 — Route Tables

What it shows: The route tables created for the project VPC.

Details visible in the screenshot:

- my-rt (rtb-0554e3ad9e75f7409) — associated with subnet-09fc9a7fb2a3679… (public-sub); not the main table
- mt-rt (rtb-0ddb9ce4637ef0f35) — 2 explicit subnet associations; not the main table
- Two unnamed main route tables are also listed (rtb-056ebe06258c16216 and rtb-0dcf2c0cccd35fa19)
- The individual routes (targets such as the Internet Gateway or NAT Gateway): Not visible in the provided screenshot

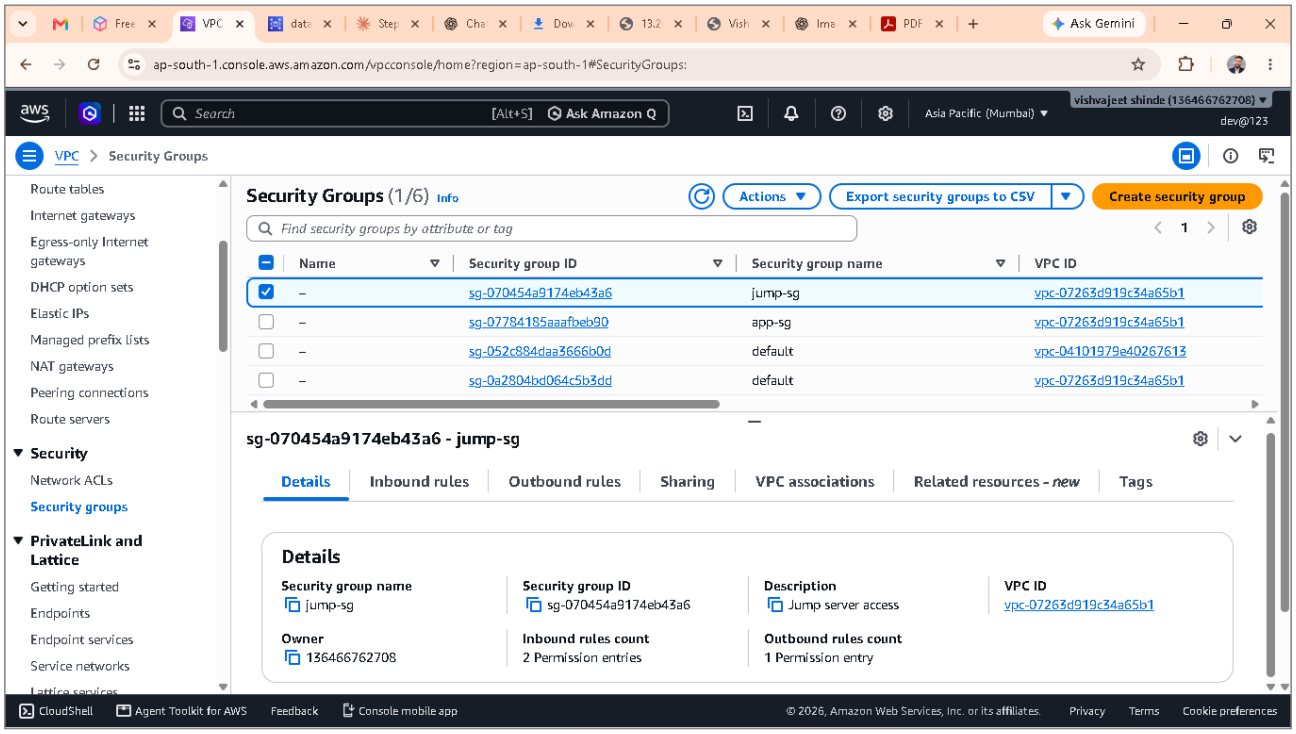


Step 6 — Security Groups

What it shows: The security groups of the project. The jump-sg group is selected.

Details visible in the screenshot:

- Security groups listed: **jump-sg** (sg-070454a9174eb43a6), **app-sg** (sg-07784185aaafbeb90) and two groups named default (6 groups in total)
- jump-sg: description “Jump server access”; VPC: my-vpc; Owner: 136466762708
- jump-sg: 2 inbound permission entries and 1 outbound permission entry
- Source CIDRs / source groups of each rule: Not visible in the provided screenshot



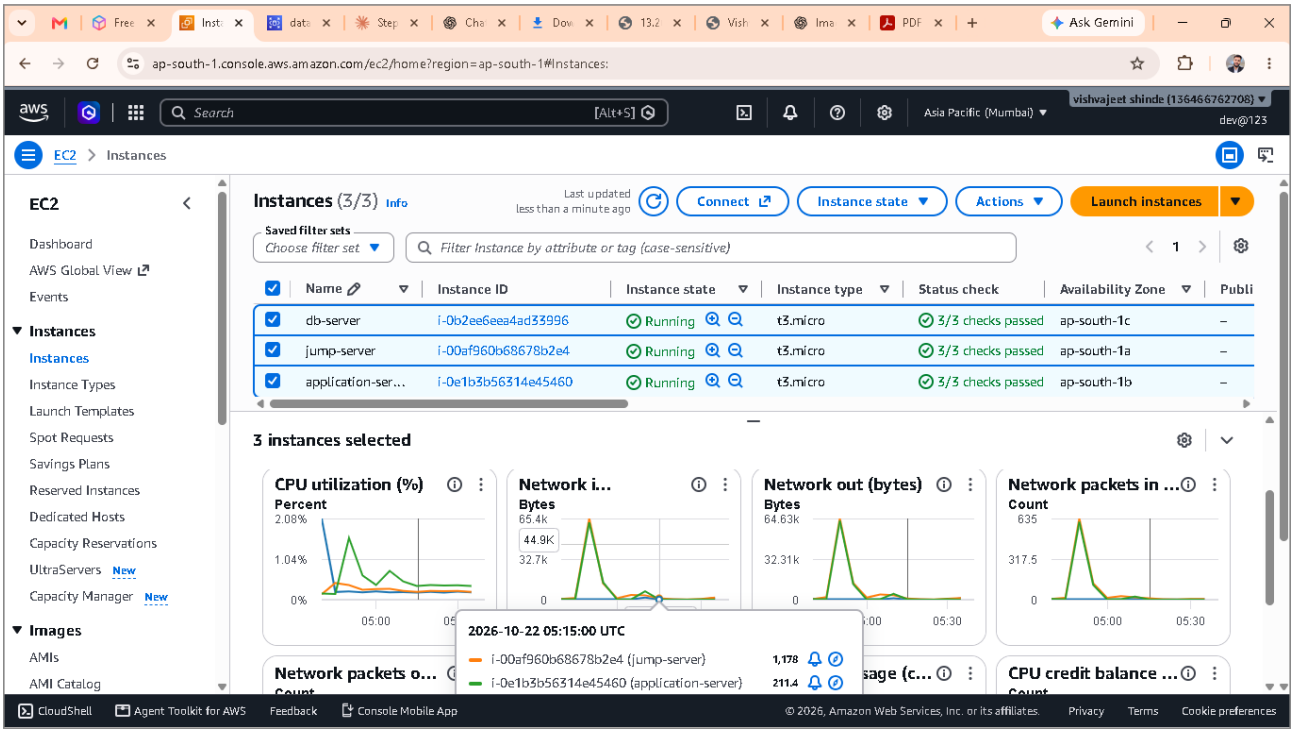
Part B — EC2 Servers and Web Software

Step 7 — EC2 Instances

What it shows: The three EC2 instances of the project, all in the running state.

Details visible in the screenshot:

- **db-server** — i-0b2ee6eea4ad33996 — t3.micro — ap-south-1c
- **jump-server** — i-00af960b68678b2e4 — t3.micro — ap-south-1a
- **application-server** — i-0e1b3b56314e45460 — t3.micro — ap-south-1b
- Instance state: Running; Status check: 3/3 checks passed for all three
- The monitoring graphs below show CPU utilization and network traffic of the three instances



Step 8 — Reaching the Application Server through the Jump Server

What it shows: The jump-server (the only public entry point) is used to log in to the application server in the private subnet.

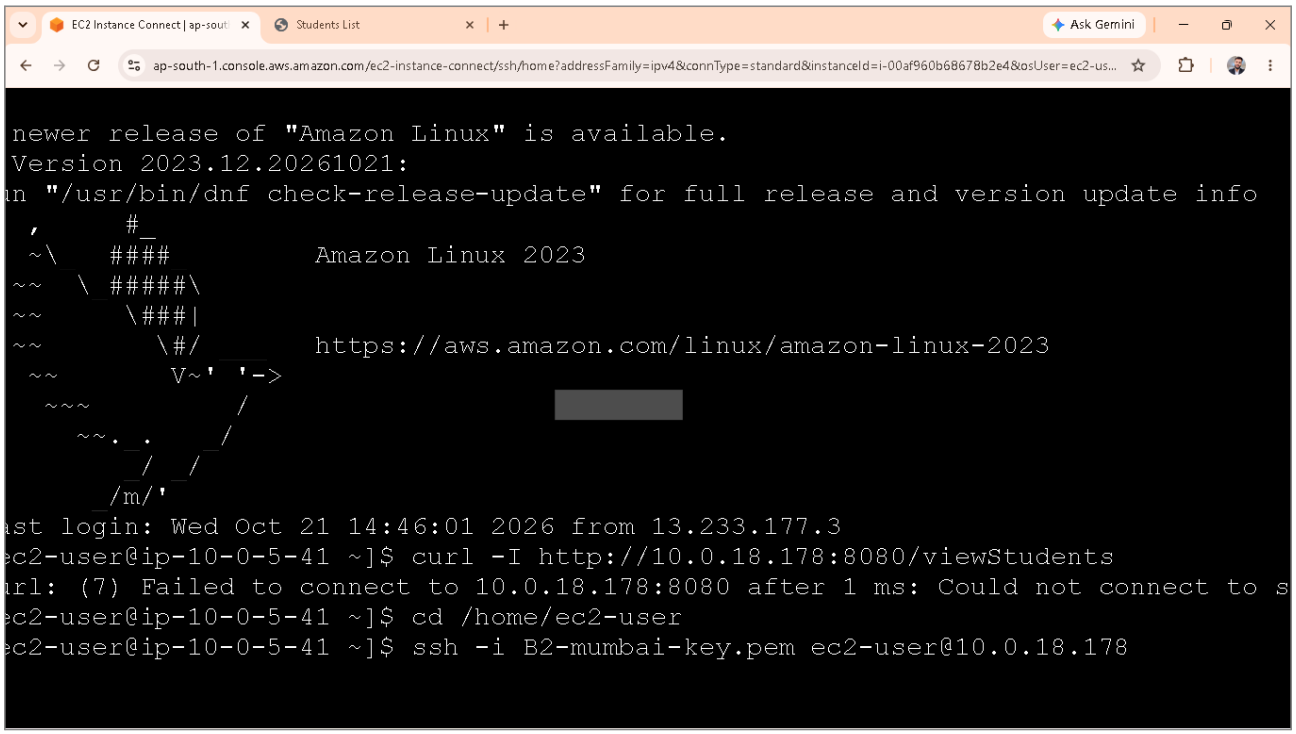
Details visible in the screenshot:

- Host: jump-server (prompt ip-10-0-5-41; public IP 65.0.26.34, private IP 10.0.5.41)
- curl to http://10.0.18.178:8080/viewStudents from the jump-server failed: “Failed to connect … Could not connect”
- After cd /home/ec2-user (where B2-mumbai-key.pem is present), ssh to ec2-user@10.0.18.178 was started to check the application server

```
curl -I http://10.0.18.178:8080/viewStudents
```

```
cd /home/ec2-user
```

```
ssh -i B2-mumbai-key.pem ec2-user@10.0.18.178
```



Step 9 — Apache Tomcat on the Application Server

What it shows: Apache Tomcat 9.0.120 is restarted and verified on the application server 10.0.18.178 (application tier).

Details visible in the screenshot:

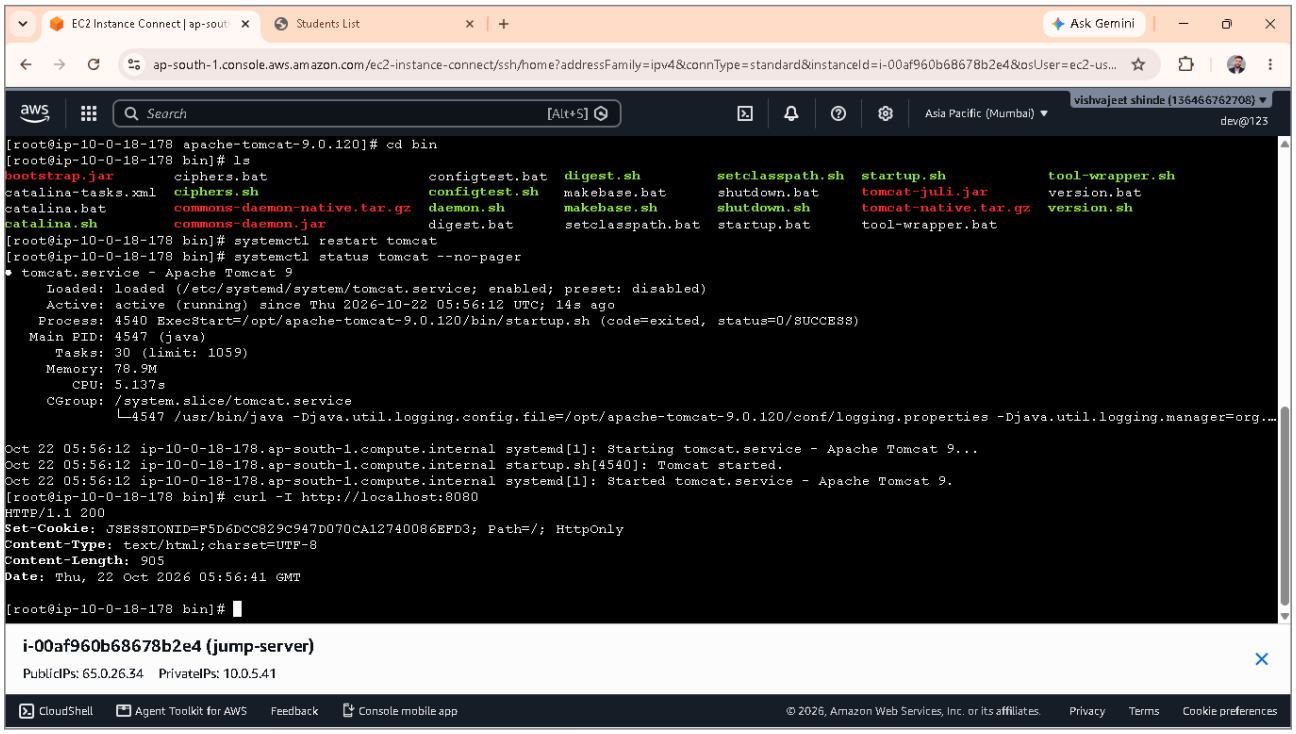
- Path: /opt/apache-tomcat-9.0.120/bin (prompt ip-10-0-18-178)
- systemctl status: tomcat.service is **active (running)** and enabled, since Thu 2026-10-22 05:56:12 UTC; Main PID 4547 (java)
- ExecStart: /opt/apache-tomcat-9.0.120/bin/startup.sh — “Tomcat started.”
- curl -I http://localhost:8080 returns **HTTP/1.1 200**

```
cd /opt/apache-tomcat-9.0.120/bin
```

```
systemctl restart tomcat
```

```
systemctl status tomcat --no-pager
```

```
curl -I http://localhost:8080
```



Step 10 — Access to the Second Private Server (10.0.34.229) via the Jump Server

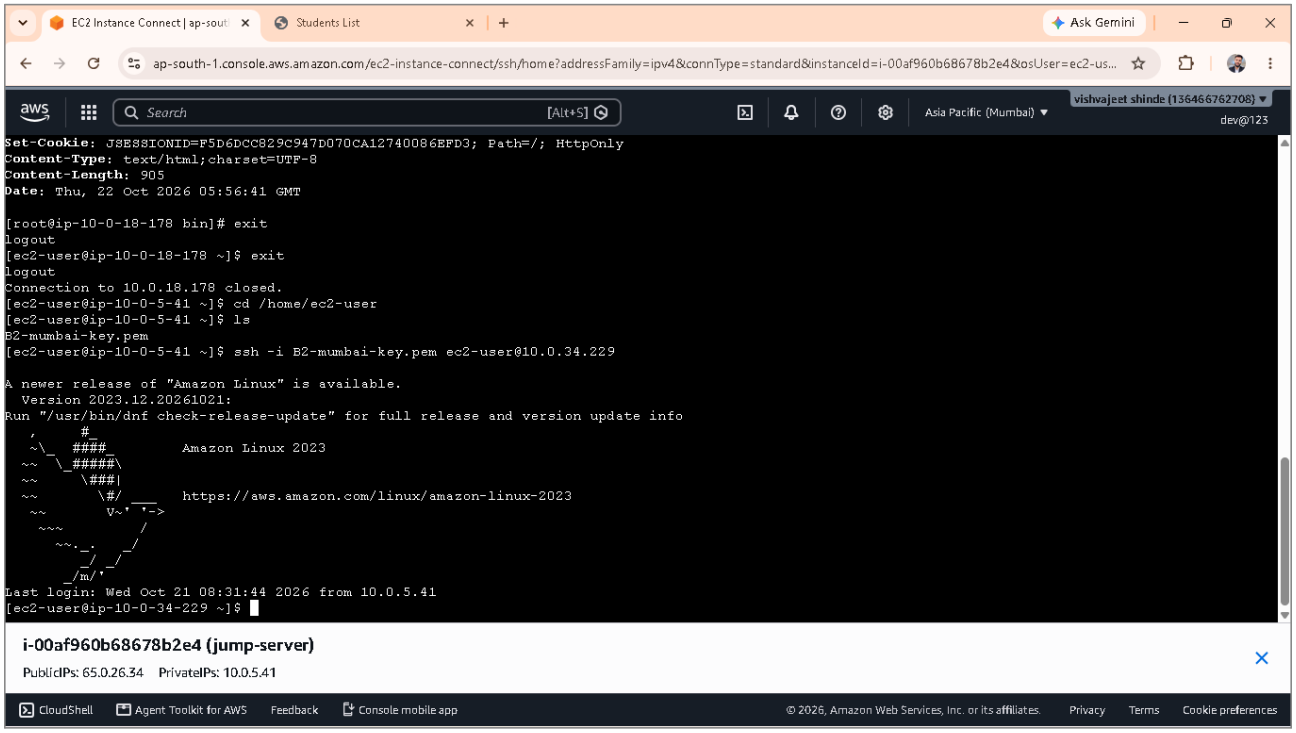
What it shows: The session leaves the application server, returns to the jump-server and connects to another private server that is used to reach the database.

Details visible in the screenshot:

- Application server session closed (exit twice); back on the jump-server (prompt ip-10-0-5-41)
- ls in /home/ec2-user shows the key file B2-mumbai-key.pem
- SSH to ec2-user@10.0.34.229 succeeds (Amazon Linux 2023; prompt ip-10-0-34-229)

```
cd /home/ec2-user
```

```
ssh -i B2-mumbai-key.pem ec2-user@10.0.34.229
```



Part C — Database (Amazon RDS MariaDB)

Step 11 — Download of the RDS Certificate Bundle (Jump Server)

What it shows: On the jump-server, after switching to root, the Amazon RDS global CA bundle is downloaded.

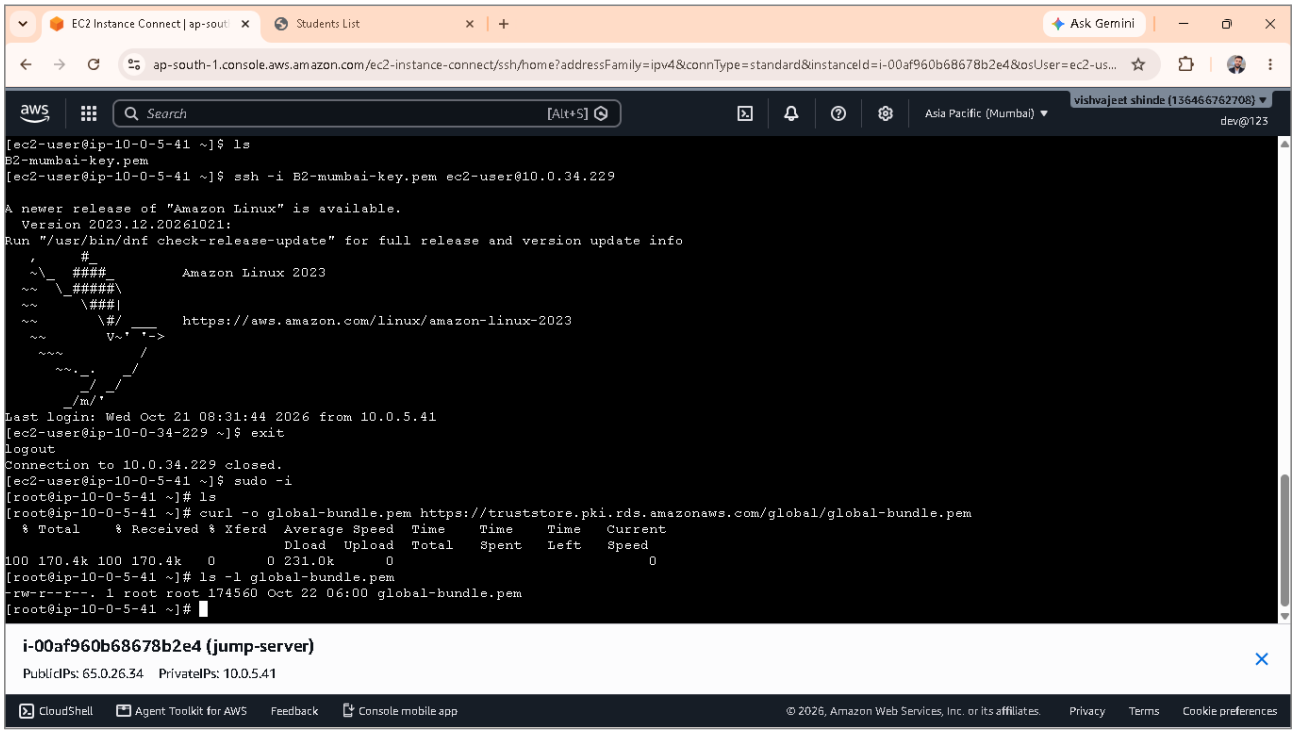
Details visible in the screenshot:

- sudo -i used on ip-10-0-5-41 (prompt changes to root)
- File saved as global-bundle.pem (170.4k downloaded; 174560 bytes in ls -l)
- Source: Amazon RDS truststore (truststore.pki.rds.amazonaws.com)

```
sudo -i
```

```
curl -o global-bundle.pem
https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
```

```
ls -l global-bundle.pem
```



Step 12 — Download of the RDS Certificate Bundle (10.0.34.229)

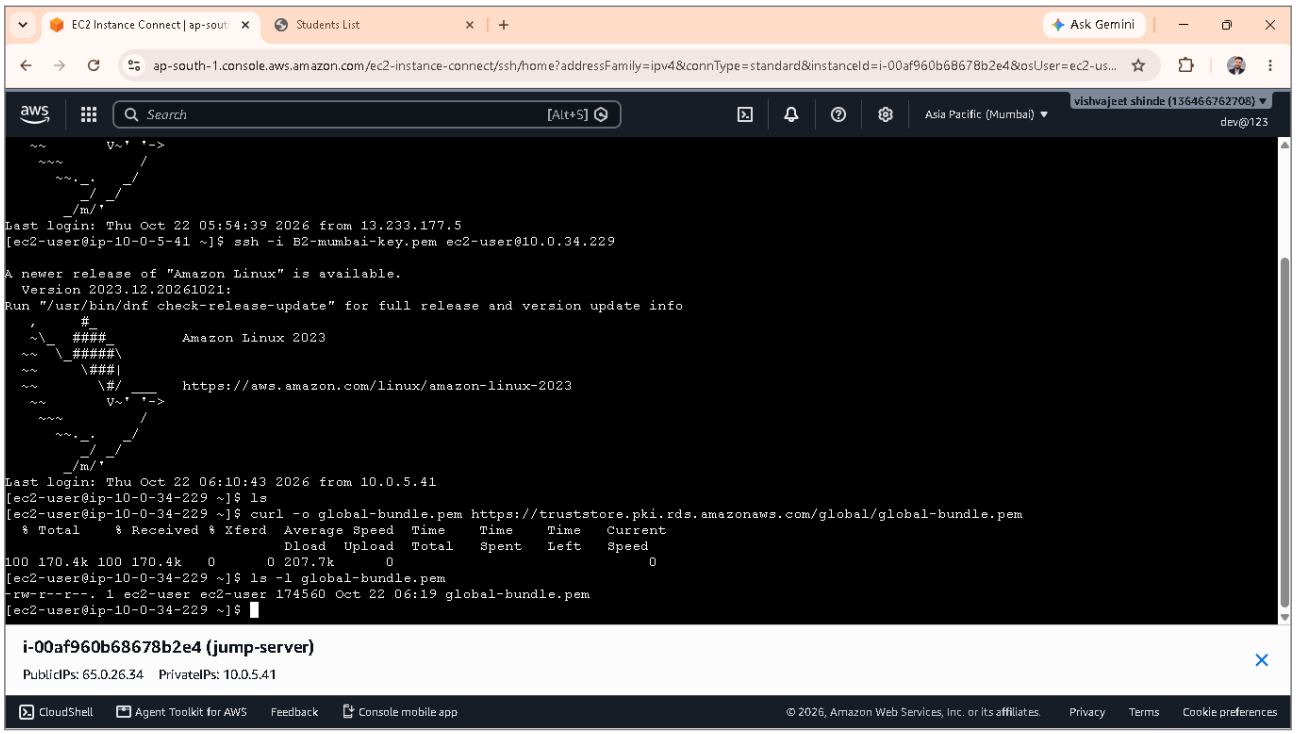
What it shows: The same certificate bundle is downloaded on the private server 10.0.34.229, so the MariaDB connection can be verified with SSL.

Details visible in the screenshot:

- Prompt: ec2-user@ip-10-0-34-229
- File saved as global-bundle.pem (170.4k downloaded; 174560 bytes in ls -l)
- Source: Amazon RDS truststore (truststore.pki.rds.amazonaws.com)

```
curl -o global-bundle.pem
https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
```

```
ls -l global-bundle.pem
```



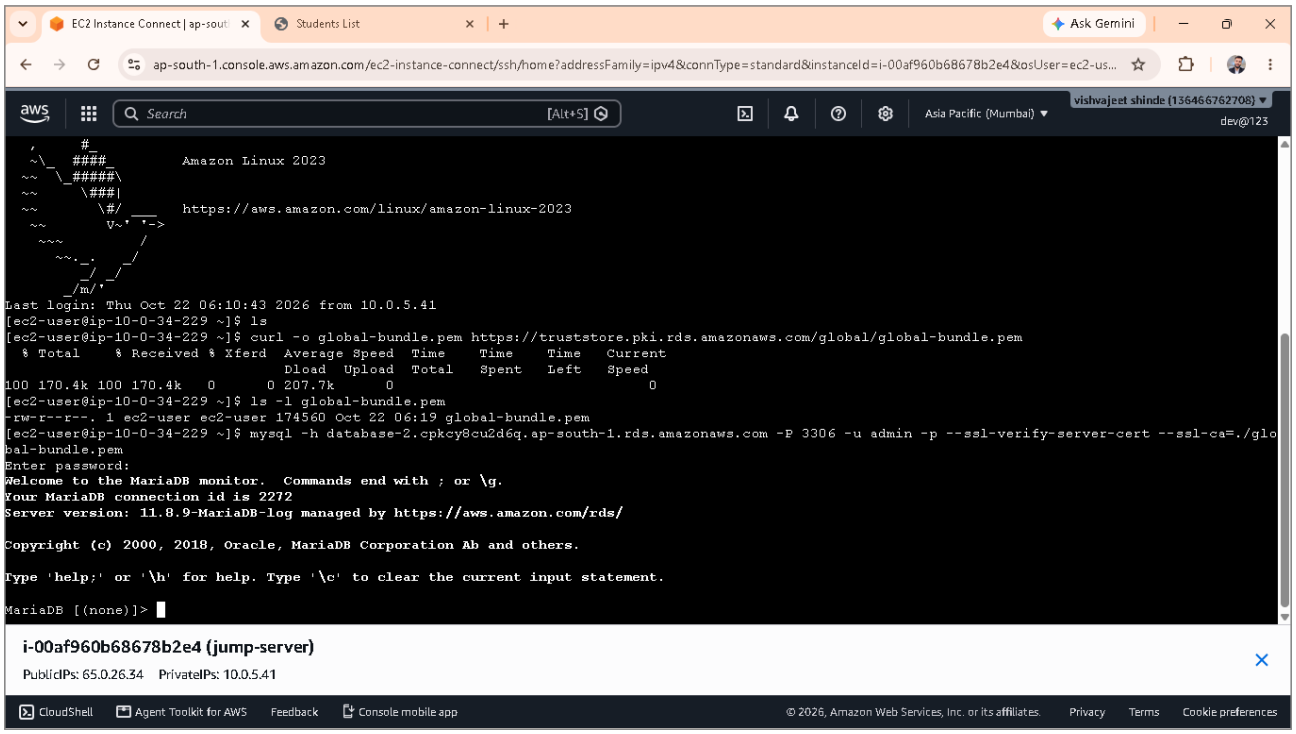
Step 13 — Connecting to MariaDB with SSL Verification

What it shows: The MariaDB client connects to the RDS endpoint using the downloaded certificate.

Details visible in the screenshot:

- Host: database-2.….ap-south-1.rds.amazonaws.com; Port: 3306; User: admin
- Options: --ssl-verify-server-cert and --ssl-ca=./global-bundle.pem
- Result: “Welcome to the MariaDB monitor”; connection id 2272
- Server version: 11.8.9-MariaDB-log, managed by Amazon RDS

```
mysql -h database-2.<endpoint>.ap-south-1.rds.amazonaws.com -P 3306 -u admin -p
--ssl-verify-server-cert --ssl-ca=./global-bundle.pem
```



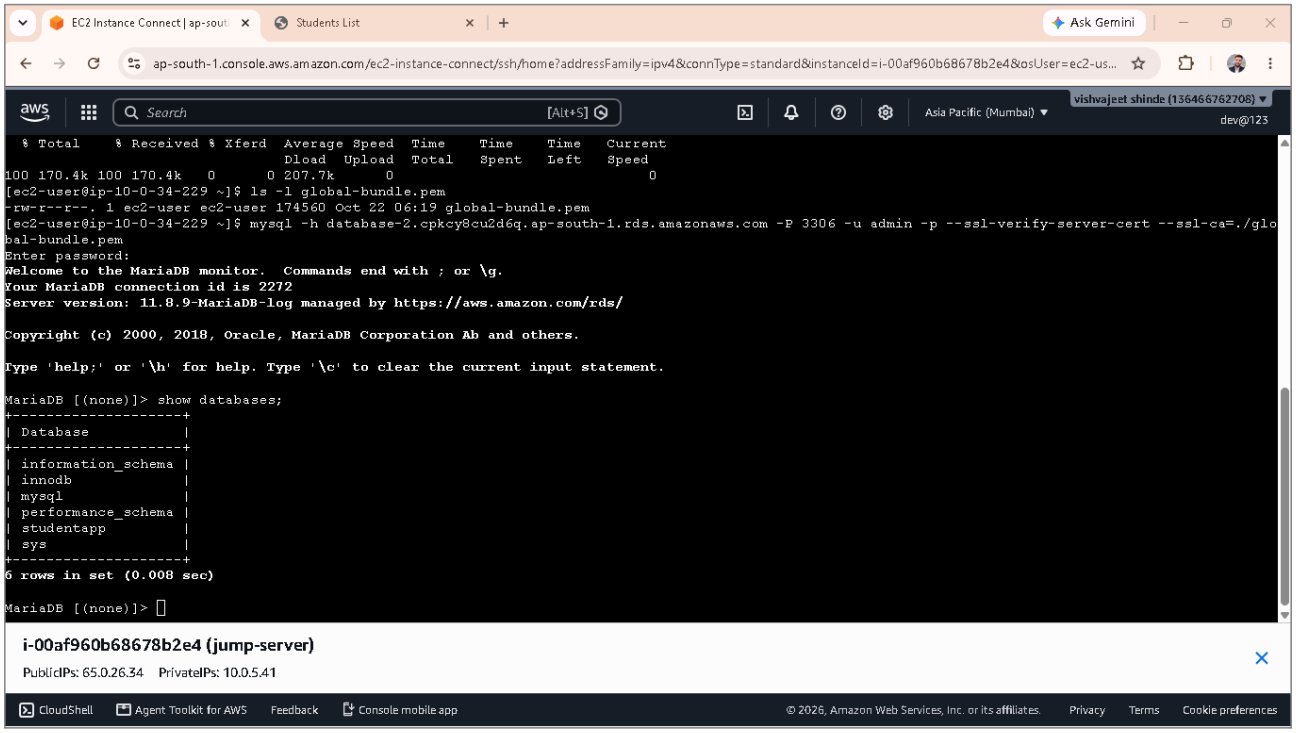
Step 14 — Listing the Databases

What it shows: The show databases; command lists what exists on the server.

Details visible in the screenshot:

- 6 rows returned: information_schema, innodb, mysql, performance_schema, **studentapp**, sys
- The application database is **studentapp**

```
show databases;
```



Step 15 — Selecting the Database and Querying the Student Data

What it shows: The studentapp database is selected, its table is listed and the data is read.

Details visible in the screenshot:

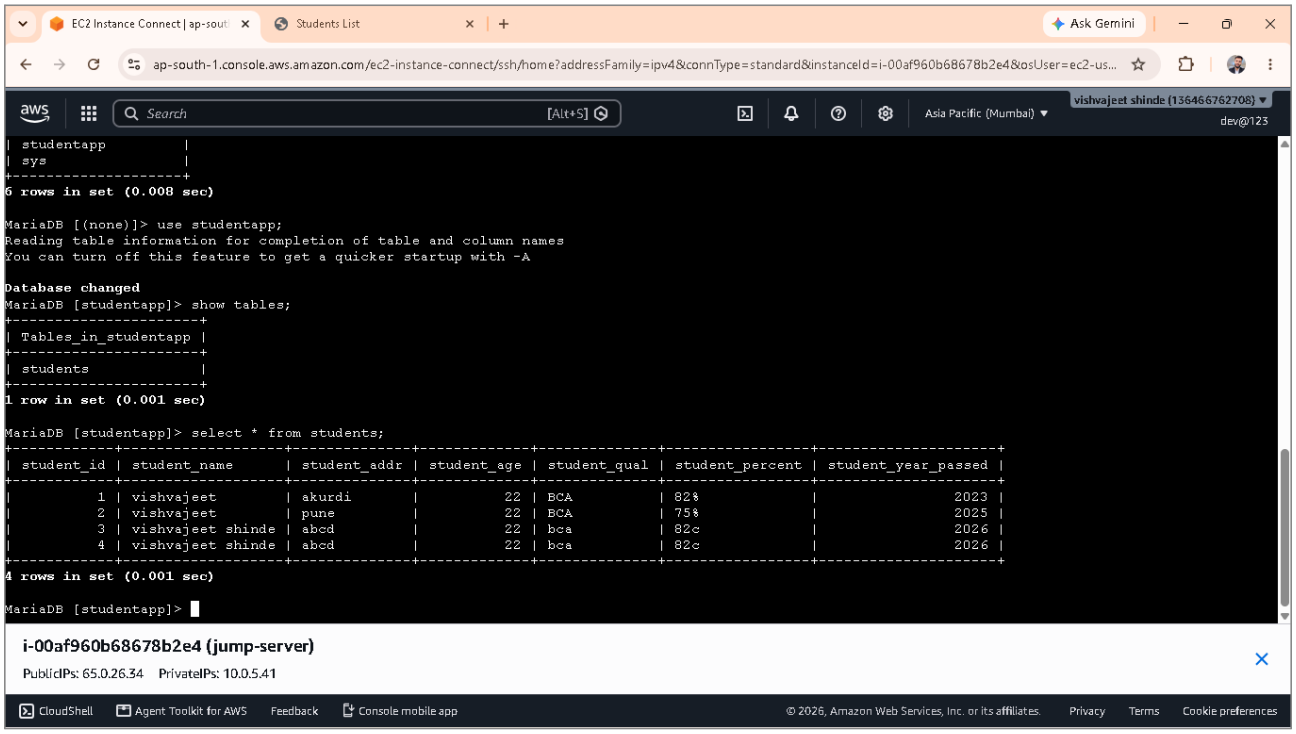
- use studentapp; -> Database changed
- show tables; -> one table: students
- select * from students; -> 4 rows returned (table below)

ID	student_name	student_addr	student_age	student_qual	student_percent	year_passed
1	vishvajeet	akurdi	22	BCA	82%	2023
2	vishvajeet	pune	22	BCA	75%	2025
3	vishvajeet shinde	abcd	22	bca	82c	2026
4	vishvajeet shinde	abcd	22	bca	82c	2026

```
use studentapp;
```

```
show tables;
```

```
select * from students;
```



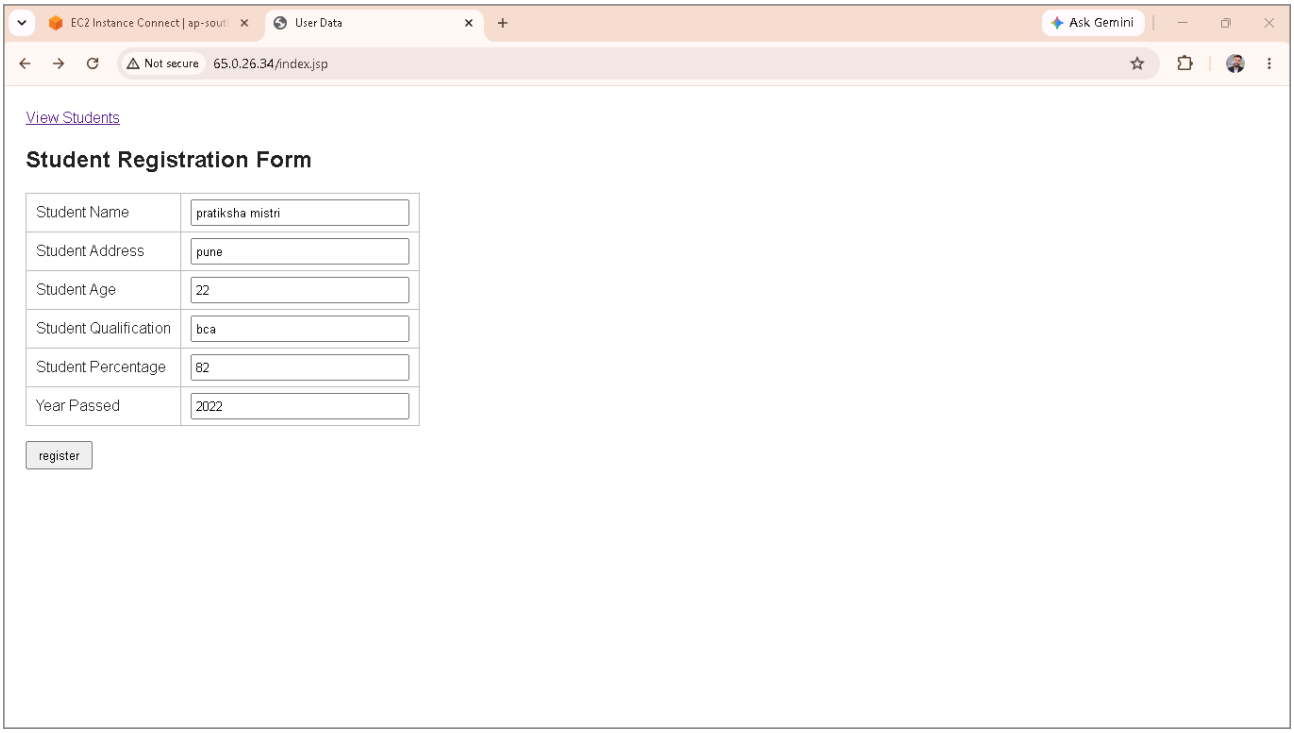
Part D — Application Output (Web Application)

Step 16 — Student Registration Form

What it shows: The web page where student details are entered.

Details visible in the screenshot:

- URL: 65.0.26.34/index.jsp (browser shows “Not secure” because the site uses HTTP)
- Fields: Student Name, Address, Age, Qualification, Percentage, Year Passed, and a register button
- Sample entry: pratiksha mistri, pune, 22, bca, 82, 2022
- A “View Students” link at the top opens the students list



Step 17 — Students List

What it shows: The page that shows the stored students.

Details visible in the screenshot:

- URL: 65.0.26.34/viewStudents
- Columns: Student ID, StudentName, Student Addrs, Student Age, Student Qualification, Student Percentage, Student Year Passed, Edit, Delete
- 5 records are shown. IDs 1 to 4 are the same rows as in Step 15; ID 5 (pratiksha mistri) is the entry registered with the form in Step 16

ID	Name	Address	Age	Qualification	Percentage	Year Passed
1	vishvajeet	akurdi	22	BCA	82%	2023
2	vishvajeet	pune	22	BCA	75%	2025
3	vishvajeet shinde	abcd	22	bca	82c	2026
4	vishvajeet shinde	abcd	22	bca	82c	2026
5	pratiksha mistri	pune	22	bca	82	2022

